

## ▼ Predictive Analytics Using Historical Data

### Forecasting Future Sales Using Linear Regression

This project aims to build a predictive model using historical sales data to forecast future trends.

## ▼ Import Libraries

Import the required libraries for data manipulation, visualization, and machine learning.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

## ▼ Load Dataset

Load the Sample Superstore Sales dataset and inspect its structure.

```
df = pd.read_excel('Sample Superstore Sales.xlsx')

df.head()
```

|   | Row ID | Order ID | Order Date | Order Priority | Order Quantity | Sales     | Discount | Ship Mode      | Profit    | Unit Price | ... | Zip Code | State      | Region  | Customer Segment |
|---|--------|----------|------------|----------------|----------------|-----------|----------|----------------|-----------|------------|-----|----------|------------|---------|------------------|
| 0 | 1      | 3        | 2010-10-13 | Low            | 6              | 261.5400  | 0.04     | Regular Air    | -213.250  | 38.94      | ... | 60035    | Illinois   | Central | Small Business   |
| 1 | 2      | 6        | 2012-02-20 | Not Specified  | 2              | 6.9300    | 0.01     | Regular Air    | -4.640    | 2.08       | ... | 98026    | Washington | West    | Corporate        |
| 2 | 3      | 32       | 2011-07-15 | High           | 26             | 2808.0800 | 0.07     | Regular Air    | 1054.820  | 107.53     | ... | 98387    | Washington | West    | Corporate        |
| 3 | 4      | 32       | 2011-07-15 | High           | 24             | 1761.4000 | 0.09     | Delivery Truck | -1748.560 | 70.89      | ... | 98387    | Washington | West    | Corporate        |
| 4 | 5      | 32       | 2011-07-15 | High           | 23             | 160.2335  | 0.04     | Regular Air    | -85.129   | 7.99       | ... | 98387    | Washington | West    | Corporate        |

5 rows × 23 columns

## ▼ Data Exploration

Understand the dimensions, data types, and summary statistics of the dataset.

```
df.shape

df.info()

df.describe()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8399 entries, 0 to 8398
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 8399 non-null  int64
1   Order ID               8399 non-null  int64
2   Order Date             8399 non-null  datetime64[ns]
3   Order Priority          8399 non-null  object
4   Order Quantity         8399 non-null  int64
5   Sales                  8399 non-null  float64
6   Discount               8399 non-null  float64
7   Ship Mode              8399 non-null  object
8   Profit                 8399 non-null  float64
9   Unit Price             8399 non-null  float64
10  Shipping Cost          8399 non-null  float64
11  Customer Name          8399 non-null  object
12  City                   8399 non-null  object
13  Zip Code               8399 non-null  int64
14  State                  8399 non-null  object
15  Region                 8399 non-null  object
16  Customer Segment       8399 non-null  object
17  Product Category       8399 non-null  object
18  Product Sub-Category   8399 non-null  object
19  Product Name           8399 non-null  object
20  Product Container       8399 non-null  object
21  Product Base Margin     8336 non-null  float64
22  Ship Date              8399 non-null  datetime64[ns]
dtypes: datetime64[ns](2), float64(6), int64(4), object(11)
memory usage: 1.5+ MB

```

|              | Row ID      | Order ID     | Order Date                       | Order Quantity | Sales        | Discount    | Profit        | Unit Price  | Ship    |
|--------------|-------------|--------------|----------------------------------|----------------|--------------|-------------|---------------|-------------|---------|
| <b>count</b> | 8399.000000 | 8399.000000  | 8399                             | 8399.000000    | 8399.000000  | 8399.000000 | 8399.000000   | 8399.000000 | 8399.00 |
| <b>mean</b>  | 4200.000000 | 29965.179783 | 2010-12-25<br>00:40:58.578402048 | 25.571735      | 1775.878179  | 0.049671    | 181.184422    | 89.346259   | 12.83   |
| <b>min</b>   | 1.000000    | 3.000000     | 2009-01-01<br>00:00:00           | 1.000000       | 2.240000     | 0.000000    | -14140.701600 | 0.990000    | 0.49    |
| <b>25%</b>   | 2100.500000 | 15011.500000 | 2009-12-20<br>00:00:00           | 13.000000      | 143.195000   | 0.020000    | -83.315000    | 6.480000    | 3.30    |
| <b>50%</b>   | 4200.000000 | 29857.000000 | 2010-12-17<br>00:00:00           | 26.000000      | 449.420000   | 0.050000    | -1.500000     | 20.990000   | 6.07    |
| <b>75%</b>   | 6299.500000 | 44596.000000 | 2012-01-01<br>12:00:00           | 38.000000      | 1709.320000  | 0.080000    | 162.748000    | 85.990000   | 13.99   |
| <b>max</b>   | 8399.000000 | 59973.000000 | 2012-12-30<br>00:00:00           | 50.000000      | 89061.050000 | 0.250000    | 27220.690000  | 6783.020000 | 164.73  |
| <b>std</b>   | 2424.726789 | 17260.883447 | NaN                              | 14.481071      | 3585.050525  | 0.031823    | 1196.653326   | 290.354383  | 17.26   |

## ▼ Data Cleaning

Check for missing values and duplicate records to improve data quality.

```
df.isnull().sum()
```

|                      | 0  |
|----------------------|----|
| Row ID               | 0  |
| Order ID             | 0  |
| Order Date           | 0  |
| Order Priority       | 0  |
| Order Quantity       | 0  |
| Sales                | 0  |
| Discount             | 0  |
| Ship Mode            | 0  |
| Profit               | 0  |
| Unit Price           | 0  |
| Shipping Cost        | 0  |
| Customer Name        | 0  |
| City                 | 0  |
| Zip Code             | 0  |
| State                | 0  |
| Region               | 0  |
| Customer Segment     | 0  |
| Product Category     | 0  |
| Product Sub-Category | 0  |
| Product Name         | 0  |
| Product Container    | 0  |
| Product Base Margin  | 63 |
| Ship Date            | 0  |

dtype: int64

```
df.dropna(inplace=True)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.drop_duplicates(inplace=True)
```

## Feature Engineering

Convert the Order Date column into datetime format and extract Year, Month, and Day features.

```
df['Order Date'] = pd.to_datetime(df['Order Date'])
```

```
df['Year'] = df['Order Date'].dt.year  
df['Month'] = df['Order Date'].dt.month  
df['Day'] = df['Order Date'].dt.day
```

## Feature Selection

Define the input variables and target variable (Sales).

```
X = df[['Order Quantity',  
        'Discount',  
        'Profit',  
        'Unit Price',  
        'Shipping Cost',  
        'Year',  
        'Month',  
        'Day']]
```

```
y = df['Sales']
```

## ▼ Train-Test Split

Split the dataset into training and testing sets.

```
X = df[['Order Quantity',
        'Discount',
        'Profit',
        'Unit Price',
        'Shipping Cost',
        'Year',
        'Month',
        'Day']]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

## ▼ Linear Regression Model

Train a Linear Regression model to predict future sales.

```
model = LinearRegression()

model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?  
LinearRegression()

## ▼ Predictions

Use the trained model to predict sales values.

```
y_pred = model.predict(X_test)
```

## ▼ Model Evaluation

Evaluate the model using MAE, RMSE, and R<sup>2</sup> Score.

```
print("MAE:", mean_absolute_error(y_test, y_pred))

print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

print("R2 Score:", r2_score(y_test, y_pred))
```

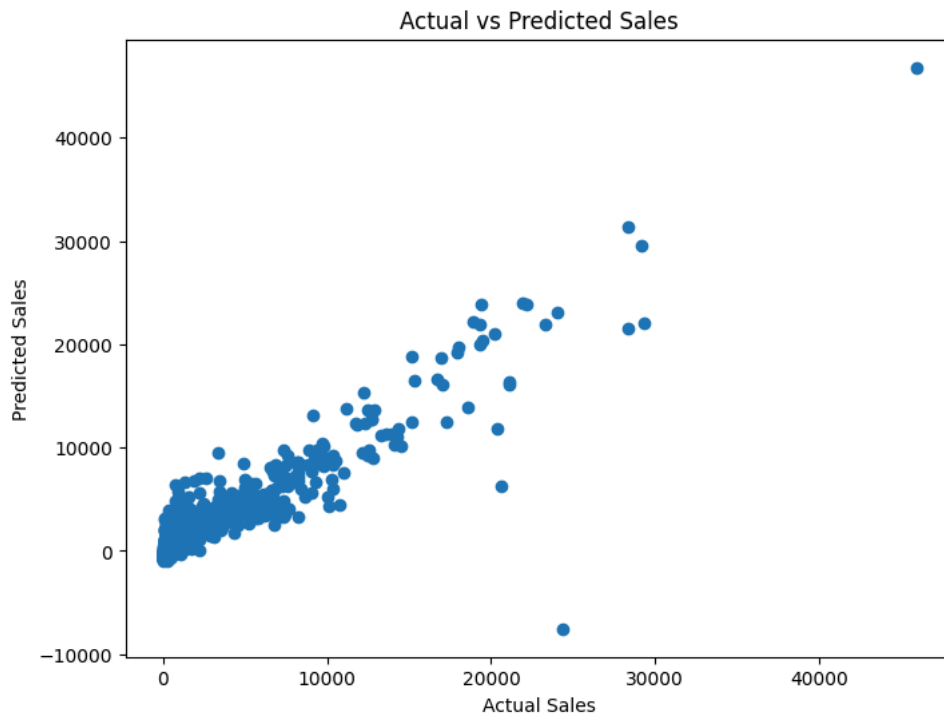
```
MAE: 713.8631327481035
RMSE: 1376.17957723261
R2 Score: 0.8566756340327351
```

```
plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")

plt.show()
```



```
future_data = pd.DataFrame({
    'Order Quantity':[10],
    'Discount':[0.05],
    'Profit':[200],
    'Unit Price':[50],
    'Shipping Cost':[15],
    'Year':[2015],
    'Month':[1],
    'Day':[15]
})

future_sales = model.predict(future_data)

print("Predicted Sales:", future_sales)

Predicted Sales: [996.70777527]
```

## Actual vs Predicted Sales

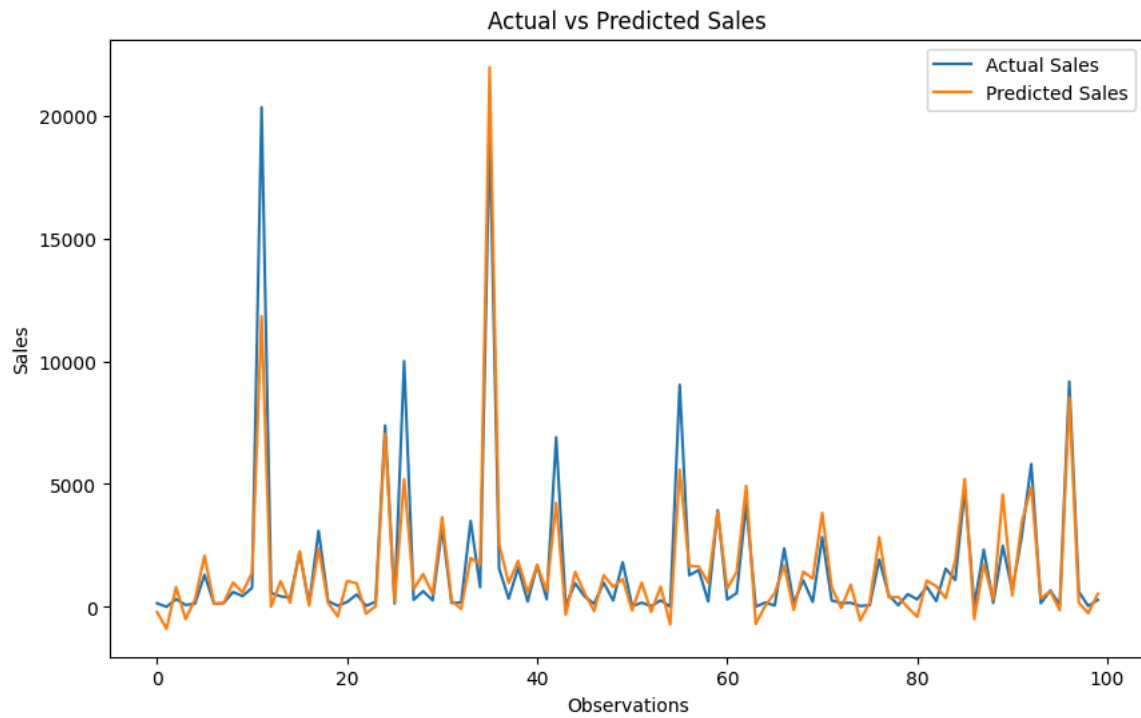
Compare actual sales values with predicted sales values.

```
plt.figure(figsize=(10,6))

plt.plot(y_test.values[:100], label='Actual Sales')
plt.plot(y_pred[:100], label='Predicted Sales')

plt.xlabel('Observations')
plt.ylabel('Sales')
plt.title('Actual vs Predicted Sales')
plt.legend()

plt.show()
```



## Scatter Plot

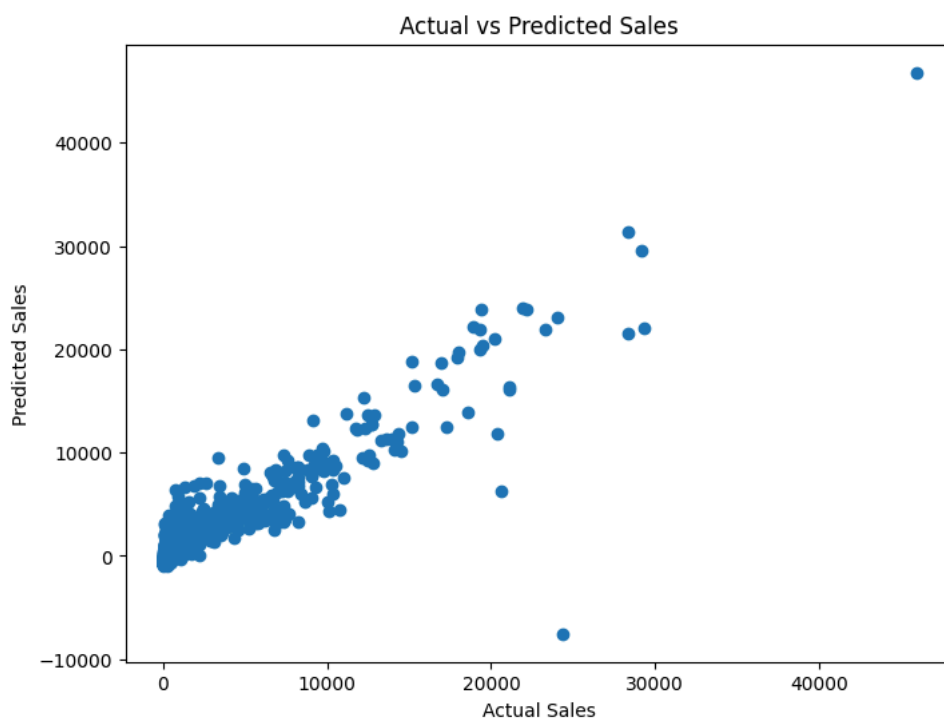
Visualize the relationship between actual and predicted sales values.

```
plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")

plt.show()
```



```
future_data = pd.DataFrame({
    'Order Quantity':[10],
    'Discount':[0.05],
    'Profit':[200],
    'Unit Price':[50],
```

```
'Shipping Cost':[15],
'Year':[2015],
'Month':[1],
'Day':[15]
})

future_sales = model.predict(future_data)

print("Predicted Sales:", future_sales)
```

```
Predicted Sales: [996.70777527]
```

## ▼ Save Predictions

Store the actual and predicted sales values in an Excel file.

```
results = pd.DataFrame({
    'Actual Sales': y_test,
    'Predicted Sales': y_pred
})
```